

API Gateway with Go, Redis & Docker

A Complete Developer Build Guide

Backend Go (Gin / net/http)	Cache & Rate Limit Redis	Auth JWT
Containerization Docker & Docker Compose	Architecture Microservices	Difficulty Intermediate–Advanced

1. What is an API Gateway?

An API Gateway is a server that acts as the single entry point for all client requests to your backend. Instead of clients talking to each individual microservice, every request flows through the gateway, which then routes it to the right service.

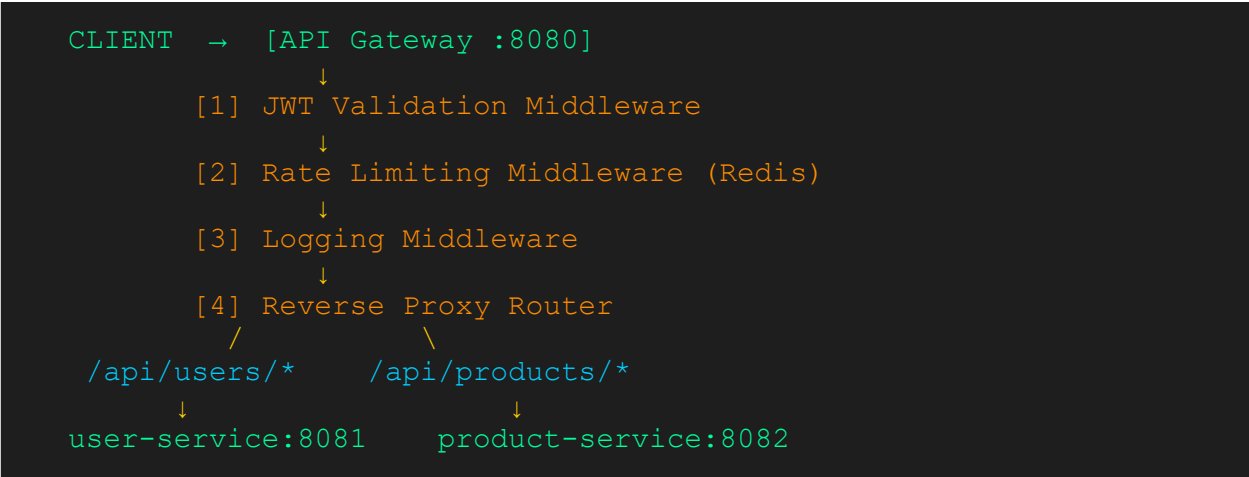
Think of it like a security checkpoint at an airport. Every passenger (request) must pass through before boarding any plane (microservice). At the checkpoint, documents are verified (JWT auth), crowd flow is managed (rate limiting), activity is logged, and you are directed to the correct gate (routing).

Why use an API Gateway?

- Single entry point — clients only need to know one URL
- Centralized security — authentication enforced in one place
- Rate limiting — prevents abuse without touching service code
- Request logging — unified observability across all services
- Decoupling — backend services can change without affecting clients

2. System Architecture

Below is the complete request lifecycle through the gateway. Every HTTP request follows this exact path:

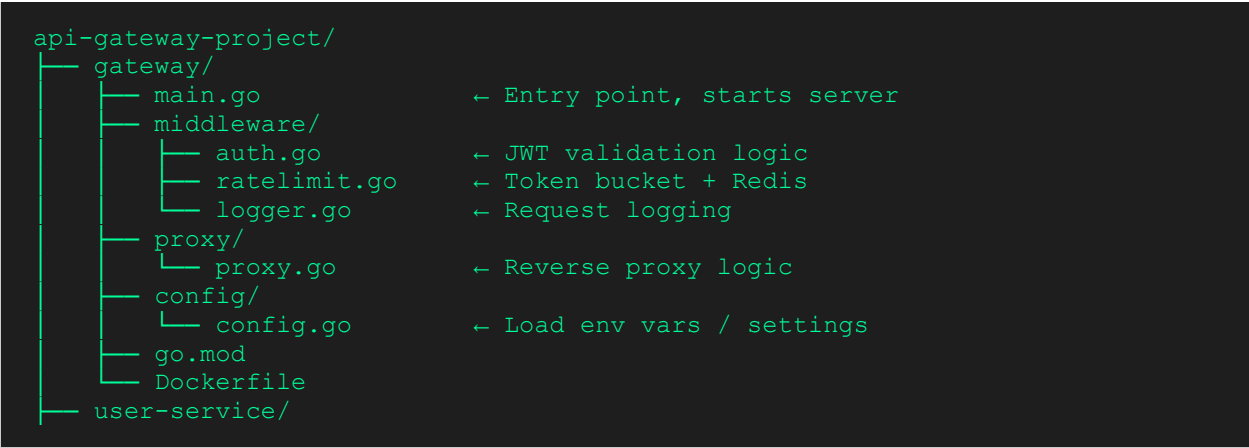


Component Roles

Component	Technology	Purpose
API Gateway	Go application (Gin)	Entry point, middleware chain, reverse proxy
User Service	Go application (Gin)	Handles user CRUD operations on port 8081
Product Service	Go application (Gin)	Handles product CRUD operations on port 8082
Redis	Redis 7 container	Stores token bucket state for rate limiting
Docker Compose	Orchestration tool	Networks and starts all containers together

3. Project Folder Structure

Organize your project exactly like this before writing any code:



```
├── main.go           ← Simple REST API
├── go.mod
├── Dockerfile
├── product-service/
│   ├── main.go       ← Simple REST API
│   ├── go.mod
│   └── Dockerfile
└── docker-compose.yml ← Orchestrates all containers
```

4. Prerequisites & Setup

Software to Install

Tool	How to Install / Verify
Go 1.21+	go.dev/download → verify: go version
Docker Desktop	docker.com/get-started → verify: docker --version
Git	git-scm.com → verify: git --version
VS Code (recommended)	code.visualstudio.com + Go extension
Postman or curl	For testing API endpoints

Initialize the Project

Run these shell commands in your terminal to bootstrap the project:

```
mkdir api-gateway-project && cd api-gateway-project
mkdir gateway user-service product-service
cd gateway && go mod init github.com/yourname/gateway
go get github.com/gin-gonic/gin
go get github.com/golang-jwt/jwt/v5
go get github.com/redis/go-redis/v9
cd ../user-service && go mod init github.com/yourname/user-service
go get github.com/gin-gonic/gin
cd ../product-service && go mod init github.com/yourname/product-service
go get github.com/gin-gonic/gin
```

5. Building the Gateway — Step by Step

Step 1: Build the Backend Microservices First

Start with the simple services so you have something to proxy to. Both services are identical in structure — just different routes.

user-service/main.go

```
package main

import (
    "net/http"
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()

    r.GET("/users", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
            "users": []gin.H{
                {"id": 1, "name": "Alice"},
                {"id": 2, "name": "Bob"},
            },
        })
    })

    r.GET("/users/:id", func(c *gin.Context) {
        id := c.Param("id")
        c.JSON(http.StatusOK, gin.H{"id": id, "name": "Alice"})
    })

    r.Run(":8081")    // listens on port 8081
}
```

Create product-service/main.go with the same structure but use /products routes and port :8082.

Step 2: JWT Authentication Middleware

JWT (JSON Web Token) is a compact, URL-safe token format. It has three base64-encoded parts separated by dots:

```
eyJhbGciOiJIUzI1NiJ9 . eyJ1c2VySWQOiIiMjMifQ . <signature>
  HEADER (algo)      PAYLOAD (data)      SIGNATURE
```

The gateway validates the signature on every request using a secret key. If invalid or missing, it returns 401 Unauthorized immediately.

gateway/middleware/auth.go

```
package middleware

import (
    "net/http"
    "strings"
    "github.com/gin-gonic/gin"
)
```

```

    "github.com/golang-jwt/jwt/v5"
)

var jwtSecret = []byte("your-secret-key-here")

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // 1. Get the Authorization header
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {
            c.AbortWithStatusJSON(http.StatusUnauthorized,
                gin.H{"error": "missing authorization header"})
            return
        }

        // 2. Strip the "Bearer " prefix
        tokenStr := strings.TrimPrefix(authHeader, "Bearer ")

        // 3. Parse and validate the token
        token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
            // Ensure the signing method is HMAC
            if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, jwt.ErrSignatureInvalid
            }
            return jwtSecret, nil
        })

        if err != nil || !token.Valid {
            c.AbortWithStatusJSON(http.StatusUnauthorized,
                gin.H{"error": "invalid or expired token"})
            return
        }

        // 4. Token is valid - pass claims to next handler
        if claims, ok := token.Claims.(jwt.MapClaims); ok {
            c.Set("userID", claims["userID"])
        }
        c.Next()
    }
}

```

**Note**

Store `jwtSecret` in an environment variable (`JWT_SECRET`) in production. Never hardcode secrets.

Step 3: Rate Limiting with Token Bucket Algorithm

The Token Bucket algorithm is the most widely-used approach for rate limiting APIs. Here is how it works conceptually:

- Each user gets a "bucket" with a maximum capacity (e.g. 10 tokens)
- Every request costs 1 token
- Tokens are refilled at a fixed rate (e.g. 1 token per second)
- If the bucket is empty, the request is rejected with HTTP 429 (Too Many Requests)

Redis is used to store each user's current token count so the state persists across multiple gateway instances (distributed rate limiting).

gateway/middleware/ratelimit.go

```
package middleware

import (
    "context"
    "net/http"
    "time"
    "github.com/gin-gonic/gin"
    "github.com/redis/go-redis/v9"
)

var redisClient *redis.Client

func InitRedis(addr string) {
    redisClient = redis.NewClient(&redis.Options{Addr: addr})
}

func RateLimitMiddleware(maxTokens int, refillRate time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        // Use client IP as the rate limit key
        key := "rate:" + c.ClientIP()
        ctx := context.Background()

        // Get current token count from Redis
        tokens, err := redisClient.Get(ctx, key).Int()

        if err == redis.Nil {
            // First request from this IP - initialize bucket
            tokens = maxTokens
            redisClient.Set(ctx, key, tokens, refillRate)
        } else if err != nil {
            // Redis unavailable - fail open (allow request)
            c.Next()
            return
        }

        if tokens <= 0 {
            c.AbortWithStatusJSON(http.StatusTooManyRequests,
                gin.H{"error": "rate limit exceeded, slow down"})
            return
        }

        // Decrement token count
        redisClient.Decr(ctx, key)
        c.Next()
    }
}
```

Step 4: Request Logging Middleware

The logger records every request's method, path, response status code, and how long it took. This is essential for debugging and monitoring.

gateway/middleware/logger.go

```
package middleware

import (
    "fmt"
    "time"
    "github.com/gin-gonic/gin"
)

func LoggerMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()

        // Process request through remaining middleware/handler
        c.Next()

        // After request completes, log the details
        duration := time.Since(start)
        fmt.Printf("[GATEWAY] %s | %d | %v | %s %s\n",
            time.Now().Format("2006/01/02 15:04:05"),
            c.Writer.Status(),
            duration,
            c.Request.Method,
            c.Request.URL.Path,
        )
    }
}
```

Step 5: Reverse Proxy Router

The reverse proxy is the core of the gateway. It receives a request, changes the destination URL to point at the correct microservice, and forwards it. The client never directly communicates with the backend service.

gateway/proxy/proxy.go

```
package proxy

import (
    "net/http/httputil"
    "net/url"
    "github.com/gin-gonic/gin"
)

// NewReverseProxy creates a proxy handler for a target service URL
func NewReverseProxy(target string) gin.HandlerFunc {
    targetURL, _ := url.Parse(target)
    proxy := httputil.NewSingleHostReverseProxy(targetURL)

    return func(c *gin.Context) {
```

```
    // Update request to target the downstream service
    c.Request.URL.Host = targetURL.Host
    c.Request.URL.Scheme = targetURL.Scheme
    c.Request.Header.Set("X-Forwarded-Host", c.Request.Host)

    // Forward the request
    proxy.ServeHTTP(c.Writer, c.Request)
}
}
```

Step 6: Gateway Main — Wire Everything Together

gateway/main.go

```
package main

import (
    "time"
    "github.com/gin-gonic/gin"
    "github.com/yourname/gateway/middleware"
    "github.com/yourname/gateway/proxy"
)

func main() {
    // Initialize Redis connection
    middleware.InitRedis("redis:6379")

    r := gin.New() // Use gin.New() not gin.Default() for custom middleware

    // — Global Middleware (applied to all routes) —
    r.Use(middleware.LoggerMiddleware()) // Log every request
    r.Use(middleware.AuthMiddleware()) // Validate JWT
    r.Use(middleware.RateLimitMiddleware(10, time.Second)) // 10 req/sec

    // — Route Groups → Microservices —
    // All /api/users/* requests → user-service:8081
    r.Any("/api/users/*path", proxy.NewReverseProxy("http://user-service:8081"))

    // All /api/products/* requests → product-service:8082
    r.Any("/api/products/*path", proxy.NewReverseProxy("http://product-
service:8082"))

    // Health check (no auth required – add it before global middleware
    // or use a separate group)
    r.GET("/health", func(c *gin.Context) {
        c.JSON(200, gin.H{"status": "ok"})
    })

    r.Run(":8080") // Gateway listens on port 8080
}
```

6. Dockerizing the Project

Dockerfiles

Each service needs its own Dockerfile. They all follow the same multi-stage pattern:

gateway/Dockerfile (same pattern for all services)

```
# Stage 1: Build the binary
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o gateway .

# Stage 2: Minimal runtime image
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/gateway .
EXPOSE 8080
CMD ["/gateway"]
```

For user-service, change the binary name to user-service and EXPOSE 8081. For product-service, use product-service and EXPOSE 8082.

Docker Compose — Orchestrating All Services

docker-compose.yml

```
version: "3.9"

services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    networks:
      - gateway-net

  user-service:
    build: ./user-service
    ports:
      - "8081:8081"
    networks:
      - gateway-net

  product-service:
    build: ./product-service
    ports:
      - "8082:8082"
    networks:
      - gateway-net

  gateway:
    build: ./gateway
    ports:
```

```
- "8080:8080"
environment:
  - JWT_SECRET=your-super-secret-key
  - REDIS_ADDR=redis:6379
depends_on:
  - redis
  - user-service
  - product-service
networks:
  - gateway-net

networks:
  gateway-net:
    driver: bridge
```

**Key
Concept**

Services communicate using their service name as hostname (e.g., redis:6379, user-service:8081) because Docker Compose creates an internal DNS network.

7. Running & Testing the Project

Start Everything

```
# From the project root directory:
docker-compose up --build
```

You should see logs from all 4 containers starting up. The gateway will be available at <http://localhost:8080>.

Generate a Test JWT Token

You need a valid JWT to call any protected endpoint. Use this quick Go snippet or jwt.io to generate one:

```
// token_gen/main.go - run with: go run main.go
package main

import (
    "fmt"
    "time"
    "github.com/golang-jwt/jwt/v5"
)

func main() {
    secret := []byte("your-secret-key-here")
    claims := jwt.MapClaims{
        "userID": "123",
        "exp":    time.Now().Add(24 * time.Hour).Unix(),
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    signed, _ := token.SignedString(secret)
```

```
    fmt.Println(signed)
}
```

Test with curl

What you're testing	curl command
Health check (no auth)	curl http://localhost:8080/health
Get users (with JWT)	curl -H "Authorization: Bearer <token>" http://localhost:8080/api/users
Get products (with JWT)	curl -H "Authorization: Bearer <token>" http://localhost:8080/api/products
No token → expect 401	curl http://localhost:8080/api/users
Rate limit test (spam it)	for i in {1..15}; do curl -H "Authorization: Bearer <token>" http://localhost:8080/api/users; done

8. Core Concepts Explained

What is Reverse Proxying?

A reverse proxy sits in front of backend servers and forwards client requests to them. The client thinks it is talking to one server, but the proxy can route to many. Key difference from a forward proxy: a forward proxy hides the client, a reverse proxy hides the server.

What is JWT and Why is it Stateless?

- Traditional sessions: server stores session data in memory/DB — hard to scale
- JWT: all user info is encoded in the token itself — server just verifies the signature
- This means any gateway instance can validate a JWT without shared session storage
- The downside: you cannot invalidate a JWT before it expires (use short expiry + refresh tokens)

Token Bucket vs Sliding Window Rate Limiting

Token Bucket (this project)	Sliding Window (alternative)
Simpler to implement	More precise, harder to implement
Allows short bursts up to bucket size	Smoothly limits across rolling time window
Good for most API use cases	Better for strict fairness requirements
Redis INCR/DECR operations	Redis sorted sets for timestamps

9. Interview Questions & Answers

Prepare these answers for your viva or technical interview:

Q1 Why use an API Gateway instead of calling microservices directly?

Centralizes cross-cutting concerns like auth, rate limiting, and logging so individual services don't need to implement them. Also simplifies the client — it only needs to know one URL. Improves security because internal services are not exposed to the internet.

Q2 How does the Token Bucket algorithm work in your implementation?

Each client IP has a key in Redis storing their current token count. On each request, we check if tokens > 0, then decrement. If empty, we return 429. Redis automatically expires the key after the refill period, which refills the bucket to max capacity. This is a simplified implementation — production systems use Lua scripts for atomic check-and-decrement.

Q3 Why did you choose Redis for rate limiting instead of in-memory?

In-memory rate limiting only works for a single gateway instance. If you run multiple gateway containers for load balancing, each has its own counter and users can exceed the limit by a factor of N. Redis is a shared external store, so all instances read and write the same counter — true distributed rate limiting.

Q4 What happens if Redis goes down?

In my implementation, the middleware uses 'fail open' — if Redis is unavailable, requests are allowed through. This is safer for availability but sacrifices rate limiting. A production approach might fail closed (reject all requests) or use circuit breakers with fallback to in-memory limiting.

Q5 How would you add a new microservice to the gateway?

Add the service to docker-compose.yml on the gateway-net network, then add one line in gateway/main.go: `r.Any('/api/newservice/*path', proxy.NewReverseProxy('http://newservice:PORT'))`. No changes needed to auth or rate limiting middleware.

10. Enhancements to Impress Interviewers

Once the base project works, add these to make it production-grade:

- **Use Sony's gobreaker library to stop forwarding requests to a failing service:** Circuit Breaker
- **Add OpenTelemetry to trace a request through gateway → service:** Distributed Tracing
- **Replace REST microservices with gRPC for faster inter-service communication:** gRPC Support
- **Use Viper to reload routing config without restarting the gateway:** Config Hot Reload
- **Auto-generate API documentation from route definitions:** Swagger / OpenAPI Docs
- **Add TLS termination at the gateway using Let's Encrypt or self-signed certs:** TLS / HTTPS
- **Use a Redis Lua script for atomic check-decrement to prevent race conditions:** Atomic Rate Limiting
- **Export request counts, latency, error rates to a Prometheus dashboard:** Prometheus Metrics

Good luck with your build! Start with the microservices, then layer in the gateway features one by one.